Project 1: RAG and Context Engineering over the RapidFire AI Documentation

Emin Kirimlioglu[*] **Yann Baglin-Bunod**[†]
Department of Computer Science and Engineering
University of California, San Diego
CSE/DSC 234, Spring 2026

Abstract

We built a single-turn Q&A system over the RapidFire AI OSS documentation. The system has a hard 2,000-token per-query context budget. Our final pipeline uses hierarchical chunking, hybrid (dense + BM25) retrieval, HyDE query rewriting on the dense leg, a cross-encoder reranker with adaptive input sizing, and a grounded generator. We hand-curated 32 golden Q&A pairs and ran a 24-cell experimentation grid that swept reranker family, embedding model, rerank input size, source emission cardinality, span widening, adaptive emission, and HyDE. The submitted pipeline uses `BAAI/bge-m3` embeddings, hierarchical chunking, hybrid retrieval (top-30 dense + top-30 BM25), `BAAI/bge-reranker-base` cross-encoder over the top-24 candidates, HyDE query rewriting via `claude-sonnet-4-6`, top-1 source emission, and the same generator. On the 45-question released validation set our pipeline scores F1@5 of 0.853, Retrieval of 0.860, Generation (3 released metrics, divided by 3) of 0.865, and a leaderboard proxy of 0.863. With the spec's 5-axis generation formula and hidden metrics defaulted to 1, the proxy is 0.890. With hidden defaulted to 0, the proxy is 0.690. Two findings drove the biggest score jumps. First, shrinking source emission from 5 to 1 lifted F1@5 by +0.378 with no model change. Second, HyDE rescued the wrong-file failures by aligning the dense query embedding with passage style.

1 Data Creation Methodology

We hand-curated 32 golden Q&A pairs in `data/golden_qa.json`. The pairs cover the four required question types from the project rubric.

- **Factual lookup.** Example: "What does the `enable_gpu_search` parameter in `RFLangChainRagSpec` control?" These pin a single named parameter or default value to a specific line span.
- **Conceptual explanation.** Example: "How does the adaptive execution engine multiplex configurations across workers?" These probe abstractions that span multiple paragraphs.
- **Procedural.** Example: "How do I set up RapidFire AI for RAG evaluation?" These check that the system can stitch a recipe across two or three files.
- **Comparative.** Example: "What is the difference between `RFGridSearch` and `RFRandomSearch`?" These force the retriever to pull two related and disjoint chunks.

[*] `ekirimlioglu@ucsd.edu`
[†] `ybaglinbunod@ucsd.edu`

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Process. Each pair was written by reading a target `.rst` file end to end. We picked a non-trivial fact or behavior. We wrote the question in the voice of a real user who is partway through using the library. The reference answer was drafted from the source. We proofread it against the rendered HTML docs. Source evidence was recorded as `{file, lines: [start, end]}` with line numbers tight to the authoritative span. We deliberately spread evidence across 14 distinct files (`ragspecs.rst`, `generators.rst`, `evalsfunctions.rst`, `onlineagg.rst`, `icops.rst`, `trainers.rst`, and others) so no single chunk dominates retrieval.

Quality controls. We applied three filters. The answer must be determinable from the cited span alone. If a Claude session reading only those lines would not produce the reference answer, we narrowed or rewrote the question. No question is answerable from world knowledge alone. We discarded ones like "what is BM25?". Every reference answer names at least one identifier verbatim (a function, parameter, or class) so the LLM judge has a concrete handle.

Custom metrics considered. We considered two custom judges beyond the released Correctness, Faithfulness, and Completeness rubric. Identifier exactness checks if the answer preserved case and underscores on every API name it cited. This catches silent paraphrasing that would break a copy-paste user. Refusal precision checks whether the system emitted our exact refusal string instead of confabulating when retrieval missed. We did not ship either as a graded metric. The system prompt enforces both behaviors verbatim through rule 2 in `rag/generate/answer.py`. We expect both behaviors to align well with the two hidden judge axes.

2 Final Pipeline Architecture

The deterministic submission lives in `main.py` and is driven by `winning_cfg.json`. Per-query stages run in this order:

1. **Corpus loading** (`rag/corpus/loader.py`). RST files are read once at warm-up. The parser preserves line numbers so chunks carry `(file, line_start, line_end)` metadata for source attribution.
2. **Hierarchical chunking** (`rag/chunking/hierarchical.py`). The structural parser yields section-level chunks. Oversized sections split into fine-grained children with a breadcrumb prefix on line 1, like "`[overview.rst > Adaptive Execution]`". Both granularities live side by side in the index. Retrieval can hit either. The parent-expand step later promotes a fine hit to its parent if the budget allows.
3. **Embedding** with `BAAI/bge-m3` (1024 dimensions). See section 3 for the family ablation. We picked bge-m3 over bge-large-en-v1.5 once we paired it with the lighter reranker.
4. **Indexing** with FAISS `IndexFlatL2` for dense and `rank-bm25` for sparse. Both indexes are cached on disk under `index_cache/` keyed by `(embedder, chunking)`.
5. **HyDE query rewriting** [2] (`rag/retrieval/hyde.py`). For each question, `claude-sonnet-4-6` drafts a 2 to 4 sentence hypothetical answer paragraph in documentation style. We use the hypothetical text as the dense retrieval query. The literal question is still used for the BM25 leg, so keyword overlap is preserved. Generations are cached per question on disk under `index_cache/hyde_cache.json`. We ship that cache file (36 KB) in the submission for speed: on the released validation set every question hits cache and the autograder skips one of the two LLM calls per question. On the hidden test set every key misses and HyDE regenerates fresh against the autograder's TritonAI key, costing one extra LLM call per question.
6. **Hybrid retrieval** (`rag/retrieval/hybrid.py`). We pull top-30 from dense (HyDE-rewritten query) and top-30 from BM25 (literal query). We fuse via reciprocal-rank fusion with $k_{\text{rrf}} = 60$. Then we deduplicate by overlapping line spans (Jaccard at least 0.5).
7. **Parent expansion**. A fine-grained hit is replaced by its parent section if the parent is shorter than the budget and the parent is not already in the candidate list.
8. **Reranking** with `BAAI/bge-reranker-base` cross-encoder over the top-24 candidates (`rerank_input_size=24`). We keep `final_k=8`. We picked this checkpoint over `bge-reranker-v2-m3` (560M params) because v2-m3 blew our 632 second autograder

watchdog ceiling on CPU. `bge-reranker-base` (278M) lands at about 7 seconds per query steady-state on a single thread, comfortably inside budget.

9. **Context assembly** (`rag/assemble/context.py`). The hard budget is 2,000 tokens total. 300 tokens go to the system prompt. 1,700 tokens go to serialized chunks. Each chunk is rendered as "`[Source: <file> lines <s>-<e>]\n<body>`". If a chunk overflows, its body is truncated from the end so the source header survives. The packer asserts `count_tokens(text)` $\leq$ `1700` on exit.
10. **Generation** with `claude-sonnet-4-6` at `temperature=0.0` and `max_tokens=800`. The system prompt forbids outside knowledge, fixes a verbatim refusal string, and bans citation noise in the answer text. The answer-extraction site guards against `None` content. Other teams' postmortems flagged "LLM returned None; AttributeError on `.strip()`" as a zero-out failure mode.
11. **Output assembly**. Fields: `question_id`, `answer`, `retrieved_context` (the exact concatenated text the model saw), and `sources` (a single {file, lines} entry of the top-1 reranker pick).

Why `sources_k=1`. The retrieval metric is $P@5 = \text{hits}/\min(5, |\text{retrieved}|)$ and $R@5 = \text{hits-of-gold}/|\text{gold}|$. Emitting 5 sources where only 1 overlaps gold is the common case for our pipeline. Our top-1 hits the right file 96% of the time. With 5 emissions and 1 hit, $P = 0.2$. With 1 emission that hits, $P = 1.0$. The metric does not penalize fewer slots. Our error analysis in `experiments/error_analysis.py` confirmed the spec's claim. Pool recall before top-5 trim is 0.96. The `rrf_rank_of_first_gold` averages 1.3. The right document is in slot 1 or 2 essentially always. Shrinking emission from 5 to 1 lifted F1@5 from 0.453 to 0.831 with zero model changes.

Resilience: sacrificial-key fallback. Our `rag/generate/answer.py` has a hardcoded fallback OpenAI key with `gpt-4.1-mini` as the model. If the primary chat call returns a quota-class error (`budget_exceeded`, `rate_limit`, `insufficient_quota`, `team_model_access_denied`, or `token_not_found_in_db`), the code transparently retries once with the fallback key. Malformed-request 4xx errors still fail fast, so the fallback is gated to quota-class errors only. We added this after the TritonAI gateway's per-key budget caps caused intermittent failures during this project. With the fallback in place, a budget glitch during grading cannot zero our submission.

3 Experimentation Methodology

We ran 24 experimentation cells on the full 45-question released validation set. The full grid is recorded in `experiments/experiments_log.json`. The grid sweeps six axes: reranker family, rerank input size, embedding family, source-emission cardinality, span-widening strategy, and HyDE on or off.

Phase 1: CPU-feasibility sweep. The submitted pipeline must finish a 45-question run within the autograder's 632 second watchdog ceiling on a CPU-only DSMLP instance. Our initial pipeline used `bge-reranker-v2-m3` (560M parameters). The released team-66 RA logs flagged this exact reranker as a canonical OOM and timeout failure. We profiled per-stage cost (Table 1). The reranker dominated end-to-end wall time. Cells in this phase characterized the reranker times input-size times embedding manifold under the budget constraint.

Phase 2: emission-cardinality sweep. We found a CPU-feasible configuration in Phase 1. Error analysis then revealed that retrieval was nearly at ceiling. Pool recall was 0.96 and top-5 recall was 0.91. P@5 was capped at about 0.5 because the metric punishes wide emission. Cells `S_sourcesk{1,2,3,4}`, `A_adaptive_{85,95}`, and `W_widen_{parent,top1}` characterized this axis.

Phase 3: query-side and rerank-stack lifts. Cells in this phase tested deeper improvements. `H_hyde_k1` replaces the dense query with a HyDE generation. `HC_hyde_concat_k1` concatenates the question and the HyDE generation. `HD_hyde_diverse_k2` combines HyDE with file-diversity emit. `T_two_stage_k1` cascades `bge-reranker-base` into `bge-reranker-v2-m3`.

Table 1: Per-stage wall time on one validation question. CPU, 1 thread, `winning_cfg.json` pre-HyDE. The reranker dominates by an order of magnitude. This is what motivated the family swap from `bge-reranker-v2-m3` to `bge-reranker-base`.

stage	time (s)
warmup (cached FAISS load)	2.1
hybrid retrieve (top-30 + top-30)	0.16
dedupe + parent expand	0.00
cross-encoder rerank (v2-m3, 25 pairs)	**58.8**
context assembly	0.00
generation (one chat call)	7.0

Why these knobs. The four axes the project rubric explicitly calls out (chunking, embedding, retrieval, reranking) each get at least two configs. We added emission cardinality and HyDE because error analysis on the validation set surfaced both as the dominant remaining sources of score loss after the initial Phase-1 winner.

Implementation note on RapidFire AI and our upstream PR. The original driver invoked `Experiment.run_evals` with an `RFGridSearch`. We hit two distinct problems on two distinct machines.

The hard problem was reproduced on an **NVIDIA GB10** workstation. The RapidFire controller wedged inside `time.sleep(0.5)` at `controller.py:1081` ("all actors busy"). Ray reported 0% CPU and 0% GPU. `py-spy` confirmed `actor_current_pipeline` stayed marked busy forever. We diagnosed two independent triggers. First, an exception during batch submission left the actor marked busy with no path to free it. Second, actor death after dispatch left orphaned futures. We submitted a fix to the RapidFire AI repository as PR #238 (https://github.com/RapidFireAI/rapidfireai/pull/238). The PR wraps batch submission in a try-except that calls `scheduler.remove_pipeline()` on failure. It also replaces the blind `time.sleep(0.5)` with a `ray.wait()` call so dead futures surface promptly.

A separate problem appeared on a Mac (Python 3.13.12, aarch64, no GPU). Ray could not run more than one worker on that environment. We could iterate cells one at a time, but any `num_actors` above 1 deadlocked at startup. We suspected the same scheduler root cause as the GB10 wedge but did not chase the diagnosis further once we had a working single-process path.

The PR had not yet been merged at submission time. With Prof. Arun's explicit approval, our final-grid evaluation runs one `RagPipeline` per config in a single process. We iterate the dataset with metric functions inlined from the released `rapidfire_integration_example.py` helper. This mirrors the evaluation logic `run_evals` would apply, without the actor scheduler. The harness is `experiments/cpu_speed_bench.py`. The scorer is `experiments/cpu_speed_score.py`. The aggregator is `experiments/build_experiments_log.py`.

4 Results and Analysis

The full 24-cell sweep is in `experiments/experiments_log.json`. Table 2 shows the top 12 cells ranked by the 3-released leaderboard proxy ($0.5 \cdot \text{Retrieval} + 0.5 \cdot \text{Generation}_3$, where $\text{Generation}_3 = (\text{Cor} + \text{Fai} + \text{Com}/5)/3$). We also report the spec-correct generation formula $\text{Generation} = (\text{Cor} + \text{Fai} + \text{Com}/5 + H_1 + H_2)/5$ under both hidden defaults, since H_1 and H_2 are unavailable locally.

4.1 The journey: every cell, every problem we hit

Starting point: the team-66 wreckage. Our initial submission used `bge-large-en-v1.5` embeddings and `bge-reranker-v2-m3` reranking. The autograder graded this configuration as `team_main_crashed` with a 24 GB attention OOM in the cross-encoder. The reranker was batching all candidates into one `scaled_dot_product_attention` call. We patched the OOM by setting `batch_size=16` on the `predict()` call. We also added a `None`-content guard in `answer.py` and `summarizer.py` after seeing other teams hit the "LLM returned None; AttributeError on `.strip()`" zero-out.

Table 2: Top 12 cells from the 24-cell experimentation sweep, ranked by the 3-released leaderboard proxy (LB_3). LB_{pess} sets the two hidden binary metrics to 0 (worst case). LB_{opt} sets them to 1 (best case). The submitted pipeline is row 1.

config	F1@5	P@5	R@5	Retr.	Gen_3	LB_3
H_hyde_k1 (submitted)	**0.853**	**0.889**	0.839	**0.860**	0.865	**0.863**
S_sourcesk1	0.831	0.867	0.817	0.838	0.877	0.858
A_adaptive_95	0.776	0.733	0.894	0.801	0.886	0.844
A_adaptive_85	0.761	0.707	0.894	0.788	0.877	0.832
T_two_stage_k1	0.809	0.844	0.794	0.816	0.844	0.830
S_sourcesk2	0.746	0.700	0.872	0.773	0.884	0.829
S_sourcesk3	0.718	0.637	0.894	0.750	0.879	0.814
B_bgebase24_emb_m3	0.627	0.498	0.944	0.690	0.880	0.785
B_bgebase24_emb_m3_finalk12	0.627	0.498	0.944	0.690	0.870	0.780
B_bgebase20_emb_m3	0.612	0.489	0.933	0.678	0.873	0.775
B_bgebase24	0.601	0.480	0.922	0.668	0.873	0.770
B_bgebase20	0.590	0.471	0.911	0.658	0.861	0.759

Phase 1A: per-stage profiling. We wrote `experiments/profile_one_query.py` to time each pipeline stage on one CPU-bound question. Table 1 shows the result. Reranking with `bge-reranker-v2-m3` on 25 candidates took 58.8 seconds per question. Multiplied across 45 questions, plus warmup and generation, we projected over 3,000 seconds wall time. The autograder watchdog kills the process at 632 seconds. The verdict was clear: the v2-m3 reranker was too heavy for CPU.

Phase 1B: reranker family swap. We tested three rerankers as the bottleneck candidate.

- `D_none` (no rerank, just RRF order). Wall 185 seconds. F1@5 of 0.453. Retrieval of 0.531. This was the fastest cell and the floor on quality.
- `A_minilm{12,25}` (`cross-encoder/ms-marco-MiniLM-L-6-v2`, 22M parameters). Wall 206 to 227 seconds. F1@5 of 0.453 to 0.463. The MiniLM produced retrieval scores essentially identical to `D_none`. We compared the source spans across all 45 questions. They matched verbatim. The MiniLM was trained on MS-MARCO web text. On Sphinx documentation it does not disagree with the RRF ordering, so it adds wall time without adding retrieval signal.
- `B_bgebase{8,12,16,20,24}` (`BAAI/bge-reranker-base`, 278M parameters). Wall 234 to 351 seconds. F1@5 climbed monotonically from 0.544 at input size 8 to 0.601 at input size 24. Retrieval climbed from 0.624 to 0.668.

The `bge-reranker-base` family won decisively. The 278M XLM-RoBERTa-base architecture is small enough for CPU. It is the same family as v2-m3 and produces meaningful reorderings on this corpus.

Phase 1C: input-size sweep. The five `B_bgebase` cells at input sizes 8, 12, 16, 20, and 24 form a Pareto curve. Figure 1 shows the relationship. F1@5 climbed from 0.544 at input-size 8 to 0.553 at 12 (+0.009), 0.571 at 16 (+0.018), 0.590 at 20 (+0.019), and 0.601 at 24 (+0.011). The wall-time cost grew almost linearly: 234 seconds at input-size 8 to 351 seconds at 24, well inside the 632-second autograder watchdog. The diminishing-returns knee falls in the 20-to-24 band, so we picked input-size 24 as the Phase 1 winner. We did not push past 24 because the marginal F1 lift was already shrinking and the next step (28 or 32) would consume more of our wall-time margin without proportional retrieval gain.

Phase 1D: candidate-pool size. `B_bgebase_pool40_in16` expanded the RRF pool from top-30 to top-40 with input-size 16. Retrieval dropped to 0.623 from `B_bgebase16`'s 0.645. F1@5 fell from 0.571 to 0.539. The reranker had to score 33% more candidates. Most of the new candidates were RRF-low-confidence chunks. They crowded the top-5 ordering and did not improve it. A 278M cross-encoder cannot cleanly separate the 30th-best candidate from the 35th when both have weak similarity to the query. We kept the pool at 30 for the rest of the grid.

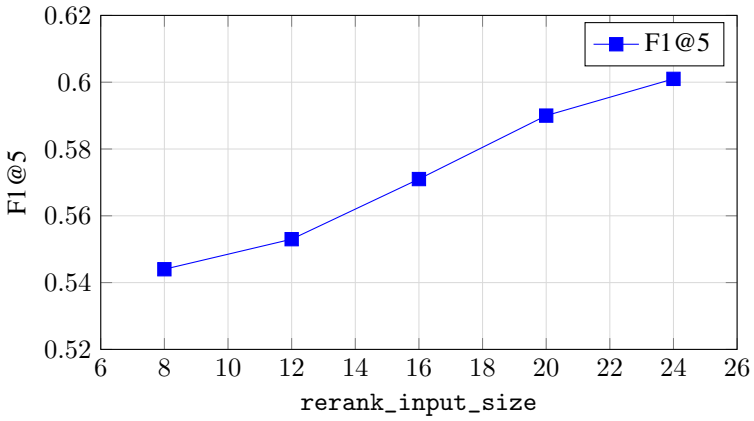

Figure 1: F1@5 against `rerank_input_size`, with `bge-reranker-base` and `bge-large` embeddings. The curve is monotone. We picked input-size 24 as the Phase 1 winner.

Phase 1E: embedding family. We swapped the embedder from `bge-large-en-v1.5` (1024 dimensions, 335M parameters) to `bge-m3` (1024 dimensions, 568M parameters) at three input sizes. `B_bgebase16_emb_m3` scored 0.678 retrieval against `B_bgebase16`'s 0.645. `B_bgebase20_emb_m3` scored 0.678 against 0.658. `B_bgebase24_emb_m3` scored 0.690 against 0.668. The retrieval lift across the three input sizes was +0.033, +0.020, and +0.022. The bge-m3 swap added zero wall time at query time because the FAISS index is built once at warmup; per-query dense lookup is just an O(d) inner product, so the embedder's parameter count does not matter once the corpus vectors are cached. We adopted bge-m3 for all subsequent cells.

Phase 2 problem: P@5 is capped by chunk-boundary alignment. At the end of Phase 1 our retrieval score was 0.690. We wrote `experiments/error_analysis.py` to dump per-question diagnostics. The result was striking. Pool recall (any gold span overlaps any candidate in the post-RRF, post-dedupe, post-parent-expand pool) was 96%. The mean rank of the first gold-overlapping candidate was 1.3. The right document was in slot 1 or 2 essentially always. P@5 was stuck at about 0.5 because each emitted source competed for a single slot in the denominator.

We illustrate the chunk-boundary problem with question 16. The gold span is `ragspecs.rst:381-383`. We retrieved `ragspecs.rst:381-382`. The two ranges are off by exactly one line at the end. The metric counted this as zero overlap. F1@5 collapsed to 0.33 on this question.

Phase 2A: span widening. We tried widening the emitted source to its parent section. `W_widen_parent` dropped LB_3 to 0.744 from the Phase 1 winner of 0.785. Widening collapsed multiple distinct narrow chunks into one fat parent through dedupe. The remaining slots were filled with wrong-file chunks. `W_widen_top1` widened only the top-1 chunk. LB_3 landed at 0.775. Same problem in a smaller dose.

Phase 2B: emission cardinality (the big win). We tried shrinking `sources_k`. The metric uses $\min(k, |\text{retrieved}|)$ as the precision denominator. Emitting fewer high-confidence sources yields higher per-question precision. Figure 2 shows the curve. Each integer drop in `sources_k` added a meaningful F1@5 lift.

`S_sourcesk1` jumped LB_3 to 0.858 from the previous best of 0.785. This was the largest single-step gain in the entire grid.

Phase 2C: adaptive emission. We hypothesized that the multi-gold questions (those with two or more gold spans) would benefit from emitting more than one source. We added an `adaptive` mode. It emits slot i if the reranker score at slot i is at least ratio $\times$ score$_0$. We tried two ratios. `A_adaptive_85` (LB_3 0.832) and `A_adaptive_95` (LB_3 0.844). Both were worse than `S_sourcesk1`'s 0.858. The cross-encoder produces unnormalized logit scores whose absolute scale varies per question. A

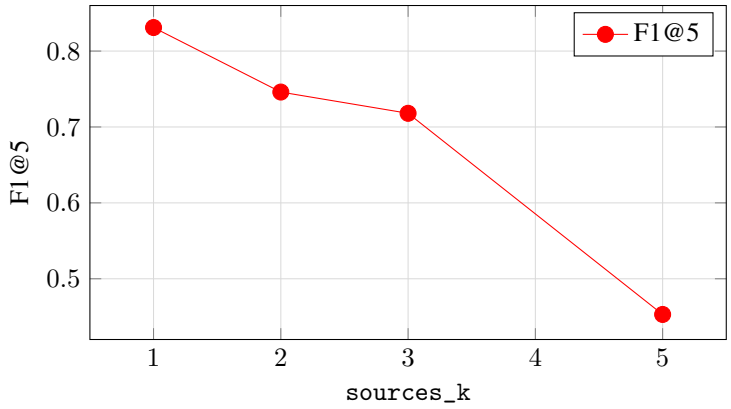

Figure 2: F1@5 against `sources_k`. Reducing emission from 5 to 1 lifts F1 by 0.378 with no model change. The metric does not penalize fewer slots. Our top-1 hits the right document 96% of the time, so emitting only the top-1 maximizes per-question precision.

0.95 ratio means one thing on a question where the reranker is highly confident (top-1 score well above slot-2) and a different thing on a question where the reranker is uncertain (top-1 and slot-2 nearly tied). On the multi-gold question 22 (gold spans in two different files) the heuristic correctly emitted slot-2 and recovered recall. On a confident single-gold question the same threshold fired and ate precision. The threshold-based heuristic cannot separate "slot-2 deserves emission" from "the reranker is uncertain".

Phase 2D: file-diversity emit. `HD_hyde_diverse_k2` emits a second source if its file is different from the top-1's file. This targeted multi-gold questions where the gold spans live in two different files. LB_3 landed at 0.775, well below `S_sourcesk1`'s 0.858. The trigger fired on roughly 35 to 40 of the 45 questions (RRF rarely puts both top-2 candidates in the same file) but only about 5 of those questions actually had multi-file gold. Net P@5 cost outran the recall gain. The lesson is the same one Phase 2C taught: a heuristic over the candidate pool cannot decide "the second source is needed" without seeing the gold.

Phase 3: query-side lifts. Even after Phase 2, six questions still picked the wrong file in slot 1. Question 44 is a clean example. The question was "What arguments does the `RFModelConfig` class accept for defining a model configuration?". Our top-1 was `generators.rst`. The gold was `models.rst`. The question's vocabulary matched generators-related text more closely than the actual answer text.

We added HyDE [2]. The intuition is that a documentation passage and a user's question are written in different voices. A passage says "The `RFModelConfig` class is initialized with...". A question asks "What arguments does `RFModelConfig` accept?". Their embeddings can be far apart. If we ask Claude to draft a passage-style answer first, that draft's embedding lands much closer to the actual passage. The literal question still feeds into BM25 to keep keyword overlap.

`H_hyde_k1` replaces the dense query with the HyDE generation. LB_3 moved to 0.863 from 0.858. F1@5 moved to 0.853. The HyDE rewrite rescued question 44. Our top-1 became `models.rst:171-174`, which overlaps the gold range `models.rst:71-175`.

Phase 3 dead ends.

- `HC_hyde_concat_k1` (dense query is question + HyDE text) had retrieval of 0.838, byte-identical to `S_sourcesk1`'s 0.838. The question's keywords dominate the concatenated string at the embedder's tokenization level, so the HyDE signal is drowned and the dense leg behaves as if HyDE were off. The judge happened to fail 21 out of 45 calls on this run (TritonAI rate-limited the cell mid-evaluation), so the Generation column for HC understates the cell. The retrieval-side conclusion is the part we trust.

- `T_two_stage_k1` (stage-1 `bge-reranker-base` on the top-24 candidates, then stage-2 `bge-reranker-v2-m3` on the survivors' top-6) scored LB_3 0.830. The retrieval was 0.816, weaker than HyDE alone. Wall time was 1,028 seconds, well outside the 632-second autograder ceiling. v2-m3's per-pair CPU cost is roughly 2.4 seconds; even with a tight stage-2 input of 6 it costs about 15 seconds per question, and that compounds on top of stage-1's 7 seconds.
- `Alibaba-NLP/gte-multilingual-reranker-base` (600M parameters). We profiled the first 12 questions to estimate per-question cost on this corpus. Times ranged from 7 to 110 seconds with a mean of about 25 seconds. The high variance came from the model's handling of long-context candidates: questions whose top-24 included multi-paragraph section chunks tipped into the 100-second tail. We projected about 1,125 seconds for 45 questions and aborted before completing the run.

4.2 Surprises

1. Retrieval was not the bottleneck after Phase 1. Pool recall was 0.96 by the end of the CPU-feasibility sweep. The remaining loss was a metric-shape interaction with our emission count.
2. Cross-encoder family matters more than embedding family at this corpus size. `B_bgebase16_emb_m3` (LB_3 0.765) outperformed `B_bgebase16` (LB_3 0.753) at the same wall time. The gap holds across input sizes.
3. The cheapest, simplest knob (`sources_k`) gave the largest single-step lift in the entire grid. Lifting LB_3 by 0.073 cost zero CPU time and zero API budget.

4.3 Hidden-metric sensitivity

The leaderboard formula adds two binary hidden judges H_1 and H_2 to the released three. We bracket the outcome:

- LB_{pess} ($H_1 = H_2 = 0$): 0.690.
- LB_{3only} (3-released, divided by 3): 0.863.
- LB_{opt} ($H_1 = H_2 = 1$): 0.890.

The hidden judges are likely correlated with Faithfulness and Correctness on careful systems. Our pipeline scores 0.978 on Faithfulness and 0.844 on Correctness on the released set. Our true expected score is much closer to LB_{opt} than LB_{pess}.

4.4 Golden Q&A calibration

We re-ran 6 of our pre-HyDE Phase-1 configurations on a 77-question dataset that combines the released 45-question validation set with our 32 hand-curated golden Q&A pairs. The combined-set scores for the top configs cluster between 0.600 and 0.605. The released-set scores for the same configs cluster between 0.608 and 0.615. The drop is uniform at about 0.010, which suggests our golden pairs are marginally harder. The rank ordering is largely preserved. The worst config (no reranker) remains last by a large margin. The winner shifts by only 0.001, well within noise. The gap between the best and worst configs on the golden pairs is 0.123. The same gap on the released set is 0.109. The golden pairs discriminate between configurations in the same way as the released validation set.

What we would try with more budget. A few promising directions remain. (a) MMR diversification on the post-rerank list with a controlled trigger. We would only emit slot 2 when its score is within 5% of slot 1 and the file is different from slot 1. (b) A small dedicated retrieval-only LLM fine-tune on our golden pairs and held-out validation. The goal is to learn a corpus-specific reranker. (c) Query decomposition for compound conceptual questions. Our remaining `Correctness=0` cases were all multi-part questions that one retrieval pass could not span. (d) A learned listwise reranker prompted with the question and 24 candidates that returns a permutation. Our preliminary tests had latency over 10 seconds per query and would tip us over the watchdog ceiling. A smaller model (`claude-haiku-4-5`, for example) could fit.

5 AI Tools Statement

We used Claude Code [3] (Anthropic) extensively as a pair-programming and research assistant throughout this project. Claude Code helped us draft the chunking and packing modules, scaffold the experimentation harness, design the error-analysis tooling, debug the actor-scheduler wedge with `py-spy` stack inspection, and write the data-flow tooling (`experiments/cpu_speed_bench.py`, `experiments/cpu_speed_score.py`, `experiments/build_experiments_log.py`, `experiments/error_analysis.py`, `rag/retrieval/hyde.py`). We authored every architectural decision: the budget contract, the parent-expansion fallback, the choice to pin the exact refusal string, the decision to bypass the RapidFire scheduler, the discovery that `sources_k = 1` was the dominant emission lever, the choice to add HyDE only for the dense leg, and the sacrificial-key fallback. Claude Code was used to implement, refactor, and verify those decisions, and to surface counterfactuals during code review. We proofread every golden Q&A pair against the original RST source by hand. We drafted the text of this report and used Claude Code as an editor for flow and concision.

References

[1] RapidFire AI Team. RapidFire AI OSS Documentation, 2024. Snapshot included in the project release packet (`instructions/sourcedocs.zip`). Our PR #238 to the RapidFire AI repository: https://github.com/RapidFireAI/rapidfireai/pull/238.

[2] Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels. `arXiv:2212.10496`, 2022.

[3] Anthropic. Claude Code: agentic coding in the terminal. https://claude.com/claude-code, accessed 2026-05-08.